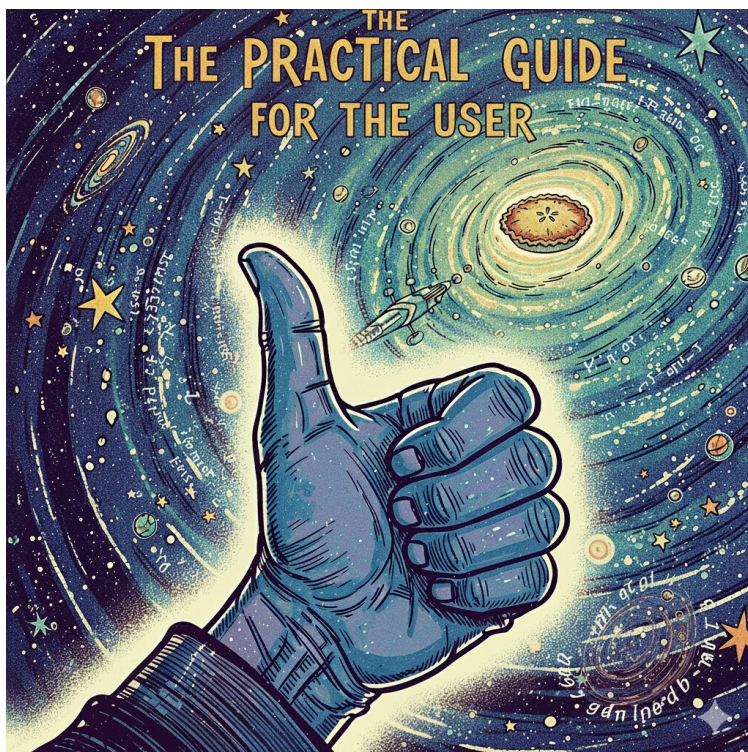


PrimePie: Number Theory Library

Practical User Guide

Jonatan Beiruty

24.7.2025



Contents

1	Introduction	2
2	Algebra	2
2.1	Additive Group Modulo n ($\mathbb{Z} / n\mathbb{Z}$)	2
2.2	Ring of Integers Modulo n ($\mathbb{Z} / n\mathbb{Z}$)	3
2.3	Primality testing in Python	4

1 Introduction

This user guide provides a practical overview of how to use the PrimePie library. Rather than focusing on the underlying mathematical theory or algorithmic design, this document is dedicated to demonstrating how to work with the library from the user's perspective.

Through concise and focused examples, users will learn how to import and interact with the various components of the library, including core number theory utilities, algebraic structures, and cryptographic primitives (in the future). Each section includes annotated code snippets and explanations to help both beginners and advanced users integrate PrimePie into their own projects.

For the theoretical background and detailed explanation of the algorithms implemented, please refer to the accompanying document.

2 Algebra

This section introduces the core abstract algebraic structures supported by the PrimePie library. We provide concrete examples of their implementations within the library.

The main structures currently supported are:

1. Groups
2. Rings

2.1 Additive Group Modulo n ($\mathbb{Z}/n\mathbb{Z}$)

The Python API exposes a thin, Pythonic wrapper over the C++ implementation. Instantiate the additive group modulo n and use the unified power method, which dispatches by argument shapes:

- `power(int base, int exponent) -> int`
- `power(Iterable[int] bases, int exponent) -> list[int]`
- `power(Iterable[int] bases, Iterable[int] exponents) -> list[int]`

Example (basic usage).

Python

```
from primepie.algebra.algebraic_structures import AdditiveModGroup

G = AdditiveModGroup(7)           # the additive group (Z/7Z, +)

print(G.identity())               # 0
print(G.inverse(3))               # 4   (since 3+4 == 0 mod 7)
print(G.operate(3, 6))            # 2   (3+6 == 2 mod 7)

# unified power (additive repeated operation)
print(G.power(2, 3))              # 6   (2+2+2 == 6 mod 7)
print(G.power([1,2,3], 2))       # [2, 4, 6]
print(G.power([1,2,3], [0,1,2])) # [0, 2, 6]
```

Notes.

- The group law is addition mod n , so `power(b, k)` means adding b to itself k times (with negative exponents handled via the additive inverse).
- All results are reduced into $0, n - 1$.

2.2 Ring of Integers Modulo n ($\mathbb{Z}/n\mathbb{Z}$)

The ring wrapper exposes both:

- The **additive group interface** (`identity`, `inverse`, `operate`, and `additive power`).
- The **ring primitives** (`zero`, `one`, `add`, `neg`, `mul`, `is_equal`, `contains`).
- A separate multiplicative power method, called `mpower`, which performs repeated multiplication under modulo n .

Example (basic usage).

Python

```
from primepie.algebra.algebraic_structures import IntegersModRing

R = IntegersModRing(7)           # the ring  $\mathbb{Z}/7\mathbb{Z}$ 

print(R.zero(), R.one())         # 0 1
print(R.add(5, 6))               # 4 (11 == 4)
print(R.neg(3))                  # 4 (-3 == 4)
print(R.mul(3, 5))               # 1 (15 == 1)
print(R.is_equal(10, 3))         # True

# additive interface inherited from Group
print(R.identity())              # 0
print(R.inverse(3))              # 4
print(R.operate(3, 6))           # 2
print(R.power(2, 3))             # 6 (2+2+2 == 6)

# multiplicative power via mpower
print(R.mpower(3, 0))            # 1
print(R.mpower(3, 4))            # 4 (3**4 = 81 == 4)
print(R.mpower([1,2,3,6], 3))    # [1, 1, 6, 6]
print(R.mpower([2,3,4,5], [0,1,2,3])) # [1, 3, 2, 6]
```

Notes.

- **Additive vs. Multiplicative Power.**
 - `power` uses the additive group structure: $a^k = a \cdots a$ (k times).
 - `mpower` uses the multiplicative structure: $a^k = a \cdots a$ (k times).
- **Negative Exponents.** Additive powers accept negative exponents (using inverses). Multiplicative powers do not (unless the structure is a field, to be added in future versions).
- **Zero Ring ($n = 1$).** In $\mathbb{Z}/1\mathbb{Z}$ we have $0 = 1$, so both `zero()` and `one()` return 0, and every product and sum is 0.

2.3 Primality testing in Python

PrimePie exposes deterministic primality tests through the ring of integers `Integers`. For convenience, you can test a single integer or a whole batch.

Single values.

Python

```
from primepie.algebra.algebraic_structures import Integers

print(Integers.is_prime(97))           # True
print(Integers.is_prime(10007))        # True
print(Integers.is_prime(10008))        # False

# Edge cases
print(Integers.is_prime(1))            # False
print(Integers.is_prime(0))            # False
print(Integers.is_prime(-11))          # False

# Large (still within 64-bit range):
print(Integers.is_prime(2305843009213693951)) # True ( $2^{61} - 1$ )
```

Batch mode. Pass a list of integers and get back a list of booleans. This uses the same deterministic test internally, and can run in parallel when PrimePie is built with OpenMP.

Python

```
nums = [2, 4, 5, 9, 1, 97, 1_000_000_007, 0, -11, 10007]
flags = Integers.is_prime(nums)
print(flags) # [True, False, True, False, False, True, True, False, False, True]
```

Filtering lists.

Python

```
data = list(range(90, 110))
is_p = Integers.is_prime(data)
primes_only = [x for x, ok in zip(data, is_p) if ok]
print(primes_only) # [97, 101, 103, 107, 109]
```

Using the precomputed prime table. If you need fast access to small/medium primes by index, use the number theory API:

Python

```
from primepie.number_theory import Primes, nth_prime, first_primes, primes_table_size

print(primes_table_size()) # e.g., 100000 (depends on build)
print(nth_prime(1), nth_prime(10)) # 2, 29
print(first_primes(10)) # [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

# Batch by indices (order preserved)
```

```
print(nth_prime([2, 4, 3, 10, 6]))    # [3, 7, 5, 29, 13]

# Slice-like access (1-based): Primes[1] == 2, Primes[1:6] -> [2,3,5,7,11]
print(Primes[1], Primes[1:6])
```

Performance tips.

- Use `Integers.is_prime` for arbitrary integers within 64-bit range; it's deterministic.
- Use `Integers.is_prime(...)` for large batches; it parallelizes when OpenMP is enabled.
- Use `nth_prime/first_primes/primes[...]` when you need many *small* primes quickly by index; these read from the precomputed binary table without loading everything into RAM.

Error handling.

Python

```
# Integers.is_prime expects int or iterable of ints.
try:
    Integers.is_prime("13")    # wrong type
except TypeError:
    print("TypeError as expected")

# The prime table uses 1-based indices:
from primepie.number_theory import nth_prime
try:
    nth_prime(0)
except ValueError:
    print("ValueError as expected (1-based indexing)")
```